

2.4 REST, CRUD and microservices

i This ticket is mostly a design/understanding task, not a big coding task. I would not change your Lambda or build API Gateway yet unless the next ticket asks you to. Your main job here is to decide what your Cold Chain API should look like and what it should return in each situation.

Okay the first things we need to understand are:

F Okay first we need to understand how APIs work:

An API is a way for two computers to talk to each other. E.G it is like using a website in your browser, but instead of clicking buttons you write code to get information from a server.

Most APIs in the world are RESTful, this has been the standard since the 2000s. They follow a set of rules and states called “Representational State Transfer”.



They organise these data resources into unique URLs (actual known as URIs uniform resource identifiers). They differentiate different types of data resources. Shown above.

A client can get a resource by making a request to that 'endpoint' over http.

The request message has a very specific format:



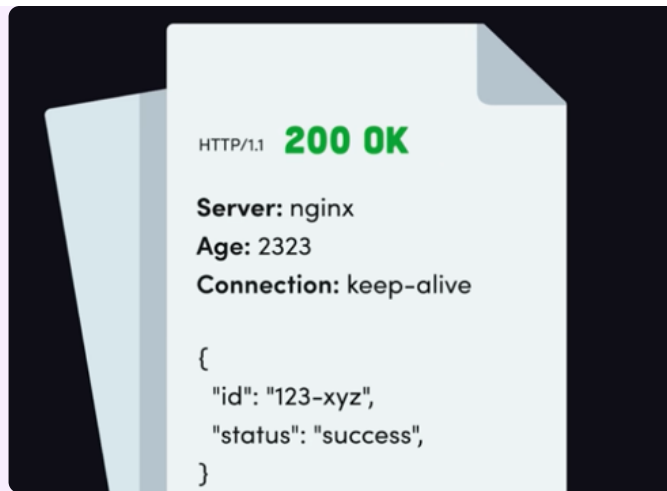
Most importantly the start line contains the an http Verb (POST, GET, DELETE etc) which signals the intent with the resources. URI that you wish to access is after the verb.

We have headers underneath this which contain metadata about the request. The header contain:

- Accept = tells server what format they want it in (e.g. Json)
- Authorisation = tell the server if the users ifs actually allowed to make that request
- Connection = controls whether the underlying network transport connection stays open or closes after the current request-response exchange completes (ASK LUKE)

Then there is the Body that contains the custom payload of data.

So when the server receives the message it will execute some code, usually reading from a database that can be formatted into a responses message.



The top of the message contains a status tell the client what happened to the request. (200, 400, 500).

then after the status code we have the response headers which tell us information about the server.

followed by the response body, which contains the response data payload and is usually formatted in JSON.

EXTRA: also to keep in mind these APIs are stateless which means the two parties don't need to store information about each other and request response cycles are independent from all other communications. Leading to well behaved web applications that are predictable and reliable.

[▶ RESTful APIs in 100 Seconds // Build an API from Scratch with Node.js Express](#)

What is an API endpoint

When you use an API, you're making a request to a server asking it to do something or get some data. But that request has to go somewhere specific it needs an **address**.

and that address is called an **endpoint**.

An API endpoint is simply a URL that represents a specific resource or functionality in an API.

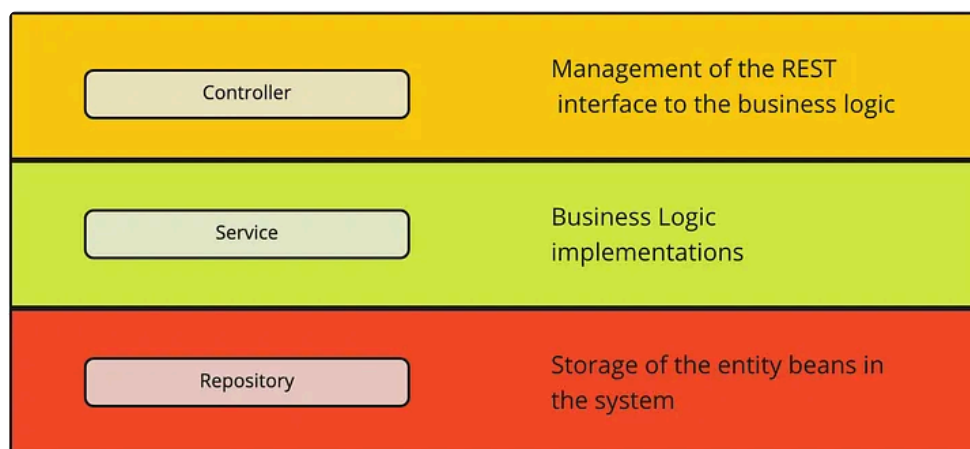
E.G think of it like a letter, the API Is the post office and the endpoint is the address that goes on the letter

▶ [What Are API Endpoints? Explained in 4 Minutes](#)

Learning objective

Controller-Service-Repository structure

This is the structure behind backend infrastructure. There are three points an API request will go through to get its desired result. These are the controller, service and repository.



Controller = The API is routed here. Imagine a controller directing the API where to go, which file to use and interact with.

Service = Is where all the code lives. This file contains the functions and code that will get the API to actually do what the request is asking it to do (e.g., if you requested a reading to be deleted).

Repository = Manages the database, and actually gets the information/data and performs what your request is asking it to do.

📌 **NOTE:** In the code example that Luke showed me, the service mentioned a coding concept called a case switch statement. What is this?

DEF: Based on a condition, `switch` selects one or more **code blocks to be executed**.

`switch` executes the code blocks that **match an expression**.

`switch` is often used as a more readable alternative to many `if...else if...else` statements, especially when dealing with multiple possible values.

```
switch( expression ) {  
    case x :  
        // code block  
        break;  
    case y :  
        // code block  
        break;  
    default:  
        // code block  
}
```

You can see that based on the condition after the case, the statements will jump straight to that section of the code and run it, rather than an IF statement where it would evaluate each condition one at a time. Additionally, there is a default at the bottom; if none of the cases are met it will jump to default and run that block of code.

Luke said: we use them instead of IF statements because IF chains can become messy in big codebases as they may need to evaluate many conditions sequentially. Therefore we utilise switch for clarity.

 NOTE: What is business logic?

DEF: Business logic is the custom set of rules, workflows, and calculations that dictate how a program processes data and makes real-world decisions.

Enforces real-world rules and policies e.g., "if a user is under 18, do not sell this item".

What does CRUD mean, and how does it map onto GET / POST / PUT / PATCH / DELETE?

DEF: CRUD stands for Create, Read, Update, and Delete. It represents the four basic functions used to manage information or data in a computer system or database.

CRUD	Meaning	HTTP method	Cold Chain example
Create	Create new data	POST	Add a new fridge reading
Read	Retrieve data	GET	Get readings
Update	Change existing data	PUT / PATCH (PUT is generally used when replacing the representation of a resource, while PATCH is intended for partial modifications.)	Change a reading
Delete	Remove data	DELETE	Delete a reading

What makes an API RESTful?

REST APIs are stateless, meaning that each request needs to include all the information necessary for processing it. In other words, REST APIs do not require any server-side sessions. Server applications aren't allowed to store any data related to a client request.

Which status code do you return when? Get properly comfortable with 200, 201, 400, 404, 409 and 500. People get these wrong constantly, and wrong codes make debugging much harder for whoever comes next

! IMPORTANT: Probably the most important part of this section

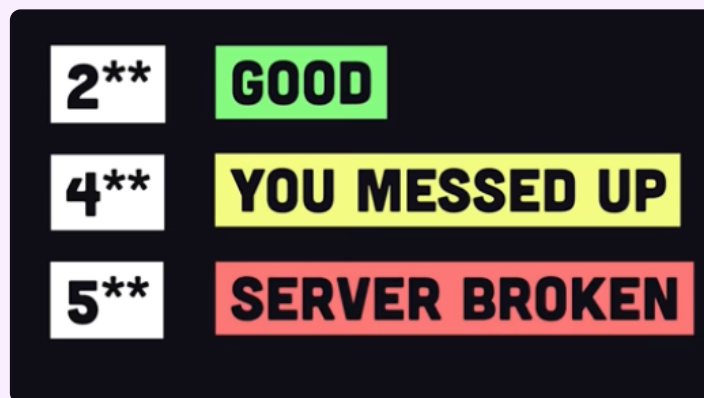
First you have to understand what a status code is:

HTTP status codes communicate whether a request succeeded and, if not, what kind of problem occurred.

Code	Beginner meaning	Cold Chain example
------	------------------	--------------------

200 OK	It worked	Successfully retrieved readings
201 Created	Something new was successfully created	New fridge reading created
400 Bad Request	The client sent invalid data	<code>maxTempC</code> is less than <code>minTempC</code>
404 Not Found	The resource you're asking for doesn't exist	Reading or branch doesn't exist
409 Conflict	Request is valid, but clashes with existing data	Reading already exists at that timestamp
500 Internal Server Error	Something unexpectedly broke on the server	Lambda crashes/DynamoDB unexpected failure

NOTE:



200's = Anything in the 200 usually means it a success

400's = Anything in the 400' usually means configuration errors

500's = Anything in the 500's usually means errors

What's the difference between 400 and 404 — and between 404 and "an empty list"?

The difference is: 400 indicates a poorly formed request causing it to fail; the resource may still exist, but the request itself is invalid. 404 means Not Found — the resource you are asking for

does not exist. The difference between 404 and an empty list is that if a collection (e.g., a table or branch) exists but has no items, a query will return a result (empty or not), so use 200. If the collection itself does not exist, you get a query error, so use 404.

What is a microservice, and what's the trade-off against one bigger service?

A microservice is a small, independent program that does one specific job in a larger software system. A big single service (monolith) puts all jobs into one huge program. There are advantages and drawbacks to both.

Benefits

- Strong Module Boundaries: Microservices reinforce modular structure, which is particularly important for larger teams.
- Independent Deployment: Small services are easier to deploy, and since they are autonomous, they are less likely to cause system-wide failures when they go wrong.
- Technology Diversity: With microservices you can mix multiple languages, development frameworks and data-storage technologies.

Cons

- Distribution: Distributed systems are harder to program, since remote calls are slow and are always at risk of failure.
- Eventual Consistency: Maintaining strong consistency is extremely difficult for a distributed system, which means everyone has to manage eventual consistency.
- Operational Complexity: You need a mature operations team to manage many services, which are redeployed regularly.

Task

i This task doesn't say in the ticket that I have to create a reading ticket. It just says to "Work through the validation rules in the brief and, for each one, decide what the API returns when it fails. Then answer these deliberately awkward questions and write your answers down." As a result I have just noted down the design of what the API should do when someone wants to add a new fridge reading.

Design the API endpoints

This comes from:

“Done when: Your full endpoint list — method, path, success code, and every failure code — is in the worksheet.”

Okay so based on the project I built so far I would design the API like this:

Create a reading

```
1 | POST /branches/{branchId}/readings
```

Success:

```
1 | 201 Created
```

Possible failures:

```
1 | 400 - Invalid reading data
2 | 404 - Branch doesn't exist
3 | 409 - Reading already exists
4 | 500 - Unexpected server error
```

Get all readings for a branch

```
1 | GET /branches/{branchId}/readings
```

Success:

```
1 | 200 OK
```

Possible failures:

```
1 | 400 - Invalid query parameters
2 | 404 - Branch doesn't exist
3 | 500 - Unexpected server error
```

If the branch exists but has no readings:

```
1 | []
```

with:

```
1 | 200 OK
```

Get readings between dates

I'd use the same collection resource with query parameters:

```
1 GET /branches/{branchId}/readings?from=2026-08-01&to=2026-08-10
```

Success:

```
1 200 OK
```

Failures:

```
1 400 - Invalid dates / from is after to
2 404 - Branch doesn't exist
3 500 - Unexpected server error
```

Again, if no readings exist during that date range:

```
1 200 OK
2 []
```

Get one reading

```
1 GET /branches/{branchId}/readings/{readingId}
```

Success:

```
1 200 OK
```

Failures:

```
1 404 - Branch or reading doesn't exist
2 500 - Unexpected server error
```

Update a reading

```
1 PATCH /branches/{branchId}/readings/{readingId}
```

Success:

```
1 200 OK
```

Failures:

```
1 400 - Invalid new values
2 404 - Reading doesn't exist
3 409 - Update clashes with another existing reading
4 500 - Unexpected server error
```

Delete a reading

```
1 DELETE /branches/{branchId}/readings/{readingId}
```

I'd normally return:

```
1 | 204 No Content
```

if you're returning no response body. `204` means the request succeeded and there is no content to send back.

If your project wants to stick to the codes listed in the ticket, you could instead use:

```
1 | 200 OK
```

with:

```
1 | {  
2 |   "message": "Reading deleted"  
3 | }
```

Failure:

```
1 | 404 - Reading doesn't exist  
2 | 500 - Unexpected server error
```

Now that the set is done we will be done, we are going to answer these 4 questions.

Question 1 = A branch exists but has no readings yet — 200 with an empty array, or 404?

Why?

It will return

```
1 | 200 OK  
2 | []
```

Because, as in other examples, the branch exists but contains no readings. Therefore it will return 200 OK because the request succeeded, and it will return `[]` because there is nothing in the branch.

Question 2 = Someone POSTs a reading for a timestamp that already exists — which code, and why not just overwrite it?

It would return

```
1 | 409 Conflict
```

This is the appropriate outcome as there is a conflict between the two readings because the timestamps are the same.

You should not overwrite the existing data because these are historical logs meant to keep records of past temperatures for the pharmacy. If you overwrite the original record, you lose the original data. For something acting as a historical log, silently changing history is dangerous.

Better response:

```
1 409 Conflict
2 "Reading already exists for this branch and timestamp."
```

Then the caller has to deliberately decide what should happen next.

Question 3 = Someone POSTs `{{maxTempC}}` lower than `{{minTempC}}` — which code, and what does the message say?

The request wouldn't make sense because maximum temp cannot be lower than minimum temp. Therefore it would return:

```
1 400 Bad Request
```

Question 4 = Someone POSTs `{{"status": "ok"}}` on a reading that's actually 15°C — what should happen, and why does this matter more than the others?

The API must not trust the client-provided status. If the measured temperatures indicate an excursion, the API should compute the correct status itself rather than trusting the incoming value. My code already enforces this as seen in the block below:

```
1 if (minTempC < 2 || maxTempC > 8) {
2   return "excursion";
3 }
4
5 return "ok";
```

So if someone was to put in

```
1 {
2   "minTempC": 3,
3   "maxTempC": 15,
4   "status": "ok"
5 }
```

it would ignore the provided status: "ok" and still output status "excursion" because the temps are outside the given acceptable range in the if statement.

Opinion

For a very small Lambda, controller-service-repository can initially feel like unnecessary extra structure because the same functionality could be written in one file with much less code.

However, as the Cold Chain application grows, separating HTTP handling, business rules and DynamoDB access will make the code easier to understand and test.



Questions / Queries / Pushback

No worries for this section so far.